

design

Cliff Pham

2360594

Intro-Computer-Networks Project 2 Design Document

Keep Alive Message Implementation :

In my current keep alive function, I am sending a keep alive message periodically every 0.1 second to all active, direct-neighbor nodes which is done by a dedicated background thread tasked with running this `keep_alive()` function . A keep alive packet is structured as follow :

```
1
2  source uuid (uuid of the sender)
3  message ("keep alive" to know it is an alive packet in listen() function)
4  backend_port (backend port of sender)
5  metric (the distance from sender to receiver)
6
```

In this `listen()` function when we detect that an incoming packet is a Keep Alive message, I immediately get the timestamp we just received this Keep Alive message, and then update it in my dictionary where the key is the uuid of the node that sent the Keep Alive message and the value associated is the **most recent timestamp** that we received this alive message.

I also added another logic case to ensure my system is robust by checking if the node that sent us this Keep Alive message is the first time we have seen this node (i.e. this uuid is not in our neighbors list of uuid's), then we add the information of this new node to our neighbors list appropriately and allow Link-State Advertisements and flooding to handle propagating this new updated graph topology. We then also update the **active** neighbor nodes list appropriately as well.

A separate `timeout_old()` background thread runs every 0.1 seconds and checks all recorded timestamps. If a node's last-seen time exceeds 10x the Keep Alive message interval, it is considered a dead node. When a timeout is detected, we **immediately** send out a Death State Advertisement to alert all neighboring nodes that the graph topology has changed and a node just died, and will flood and propagate throughout the network appropriately. Following this, we then remove the dead node from all local data structure of the current node / process. This includes any entries within the following data structures:

```
1 active_peers_uuid
2 map
3 uuid_to_last_alive
4 uuid_to_seen_seq
```

alongside with removing it from other node's adjacency list in our map data structure to minimize the risk of a stale network.

Link-State Advertisement Implementation :

There exists a background thread that runs the `link_state_adv()` function to send a Link-State packet to all active direct neighbors every 2 seconds which was what I decided to go with. A future implementation would to create a helper link state advertisement function to call this independently every time the graph topology changes from adding neighbors, since we already send an alert in the case of a dead node.

An LSA packet contains the following useful information:

```
1 message : the string "Link State Packet" to allow us to distinguish this is
a LSA packet
2 source_uuid : uuid of the sender
3 source_name : name of the sender
4 neighbors : a dictionary of uuid of neighboring node of sender to the
distance from the sender
5 seq : a sequence integer to determine a new LSA packet to update
```

When an LSA packet is received, the `listen()` thread will compare the LSA packet's sequence number to the highest previously seen sequence number from the sender and if the new sequence number in the packet we just received is higher, we update the local network map and the UUID to name table, and then flood the packets to all other active neighbors through the `link_state_flood()` function of of the current local graph structure. This is exactly how other non-direct neighbor nodes will figure out the entire graph topology, even if they are not connected directly to the current node / process we are "in". There is an optimization that we do not flood to the original sender node to save on resources and time.

Libraries used :

heapq (priority queue library) version 3.14 [link here](#)

socket version 3.14 [link here](#)

sys version 3.14 [link here](#)
ast version 3.14 [link here](#)
threading version 3.14 [link here](#)
time version 3.14 [link here](#)

Extra capabilities implemented :

I implemented a special type of message packet called "add neighbor" which is organized in the following structure :

```
1  message : "add neighbor" string to distinguish this is the special add  
   neighbor packet  
2  uuid : uuid of sender node  
3  backend_port : backend port of sender node  
4  name : name of sender node  
5  metric : distance of sender node to the node currently in 'view'
```

The purpose of this special packet is for when `addneighbor` command is called, the new node is added to the general overall neighbor nodes list, and then a direct message consisting of this special packet is then sent to the newly added peer so it can register the reverse link immediately, without waiting for the next Keep Alive message to update it. This also allows us to fill in the name element in the newly added node's UUID to name dictionary, to avoid any errors in the case an LSA packet was not sent at the moment we ran the `neighbors` command, resulting in indeterministic and timing errors. This allows the system to be robust, and handle this special edge case which could be problematic if we didn't treat it with this helper packet and logic.